# AquaTrack

## Project Architecture, Setup, Operational Guide, Technical Assessment, and Improvement Roadmap

Document Version: 1.0
Generated: June 01, 2026

Veekay Aquatech Private Limited
Classification: Internal

## Table of Contents


1. Executive Summary
2. Project Overview
3. Technology Stack
4. Architecture Analysis
5. Folder Structure Analysis
6. Codebase Assessment
7. Dependency Analysis
8. Database Analysis
9. API Analysis
10. Configuration Analysis
11. Local Development Setup Guide (Windows)
12. Deployment Analysis
13. Monitoring & Logging Review
14. Security Review
15. Performance Review
16. Refactoring Recommendations
17. Proposed Future Architecture
18. Knowledge Transfer Section
19. Team Review Checklist
20. Action Plan
21. Risk Register
22. Appendix

# 1. Executive Summary

## 1.1 Project Overview

AquaTrack is a full-fledged water bottle delivery management platform developed for Veekay Aquatech. It supports B2B operations (quick-commerce channels such as Blinkit, Zepto, and IBM) and B2C delivery workflows. The system coordinates partners, retail stores, delivery managers, delivery partners, channel administrators, and super administrators through a unified backend API, a React web portal, and a React Native mobile application.

## 1.2 Business Purpose

- Manage water bottle orders from store partners through delivery fulfillment.
- Track physical bottle assets via QR codes and lifecycle states.
- Automate recurring store orders based on consumption and channel rules.
- Provide role-based dashboards for operations, compliance, and reporting.
- Integrate with Google Sheets for operational reporting (ZEPTO channel).
- Support employee attendance and vendor licensing workflows.

## 1.3 Key Findings

| Finding | Severity | Summary |
|---|---|---|
| Production system is functional | Positive | Core order, delivery, and admin flows are implemented and deployed. |
| Minimal documentation | High | README contains one line; no .env.example or runbook existed. |
| Monolithic UI/route files | Medium | SuperAdminDashboard.jsx (~8,000 lines), partners.py (~1,600 lines). |
| Configuration fragmentation | Medium | JWT secret hardcoded; multiple auth modules; doubled API prefixes. |
| No CI/CD or automated tests | High | Deployments and quality assurance are manual. |
| Secrets in repository | Critical | google_service.json, credentials in migrate.py, default JWT secret. |
| In-process background jobs | Medium | Scheduler and Google Sheet sync run inside API process. |

## 1.4 Current Project Status

- The application is deployed to production on Fly.io (aquatrack-prod-final.fly.dev)
- The web portal is deployed via GitHub Pages.
- The mobile app is distributed via Google Play.
- Local development has been verified: the FastAPI backend imports successfully, creates SQLite tables automatically, and serves OpenAPI documentation.

### 1.5 Overall Health Assessment

| Dimension | Score (1–10) | Assessment |
|---|---|---|
| Maintainability | 4 | Large files, duplicated modules, inconsistent patterns. |
| Scalability | 3 | Single-process monolith; no cache or message queue. |
| Security | 5 | JWT auth works; secret management and credential hygiene weak. |
| Reliability | 6 | Business logic proven in production; ops tooling immature. |
| Testability | 2 | Almost no automated test coverage. |
| Documentation | 2 | This document addresses the documentation gap. |
| DevOps | 4 | Docker + Fly.io exist; no CI/CD pipeline. |

## 2. Project Overview

### 2.1 High-Level Description

AquaTrack is a three-tier monorepo consisting of a Python FastAPI backend, a React web portal for administrative and partner users, and a React Native mobile app for field operations. All clients communicate with the backend via REST APIs secured with JWT tokens.

### 2.2 Major Features

| Feature Area | Description | Primary Users |
|---|---|---|
| Order Management | Create, route, assign, deliver, and confirm water orders | Partner, DM, DP, Super Admin |
| Store Management | CRUD stores, channel assignment, auto-order configuration | Super Admin, Channel Admin |
| Partner Management | Partner CRUD, store mapping, hierarchy | Super Admin, Channel Admin |
| Delivery Routing | Auto-route orders to delivery managers; assign to delivery partners | Delivery Manager, Super Admin |
| Bottle Tracking | QR generation, assignment, collection, warehouse tracking | Super Admin, Delivery Partner |
| Auto-Ordering | Daily scheduled orders based on store consumption rules | System (scheduler) |
| Compliance | Store compliance document upload and PDF generation | Partner, Field staff |
| Complaints | Submit and resolve operational complaints | Partner, Admin, Channel Admin |
| Reports | Excel/PDF reports, monthly pivot exports, Google Sheets sync | Super Admin |
| Attendance | Employee check-in/check-out and leave | Employee |
| Vendor Licensing | State-based vendor license management | Super Admin, Public (vendor-info) |
| Authentication | Unified login, OTP signup, password recovery | All roles |

## 2.3 Core Workflows

**Order Lifecycle:**

- pending → accepted → assigned_to_manager → assigned → in_transit → delivered_pending_confirmation → delivered_confirmed
- Alternative states: cancelled, rejected, delivery_disputed

**Auto-Order Workflow:**

- APScheduler triggers daily at 6:00 AM IST.
- For each store with auto_order_enabled: validate manager + partner mapping.
- Create system order with status assigned_to_manager if rules pass.

## 2.4 User Journey

| Role | Client | Primary Journey |
|---|---|---|
| Super Admin | Web Portal | Login → Dashboard → Manage stores/partners/orders/bottles/reports |
| Partner | Web + Mobile | Login → View stores → Place orders → Confirm delivery |
| Channel Admin | Web Portal | Login → Channel-scoped dashboard → Manage partners/complaints |
| Delivery Manager | Web + Mobile | Login → View assigned orders → Monitor team |
| Delivery Partner | Mobile | Login → View assigned orders → Update delivery status |
| Employee | Web Portal | Login → Attendance check-in/out |

## 2.5 Business Processes Supported

- B2B water delivery to quick-commerce store locations (Blinkit, Zepto, IBM, General).
- Empty bottle return tracking and pending bottle counts per store.
- Monthly consumption caps and per-day consumption for automated ordering.
- Delivery partner onboarding with KYC (govt ID, vehicle, photos) and admin approval.
- Channel-segregated reporting and complaint routing.
- Operational report upload and automated Google Sheets synchronization.

## 3. Technology Stack

| Category | Technology | Version / Notes |
|---|---|---|
| Backend Language | Python | 3.11+ local; 3.12 in Dockerfile |
| Backend Framework | FastAPI + Uvicorn | Latest from requirements.txt |
| ORM | SQLAlchemy | Declarative models |
| Validation | Pydantic v2 | Some schemas still use orm_mode |
| Auth | JWT (python-jose) + bcrypt (passlib) | OAuth2PasswordBearer |
| Web Frontend | React 18 + Vite 7 | web-portal/ |
| Web Routing | React Router 7 | |
| Mobile | React Native 0.84 | Node >= 22.11.0 required |
| Mobile Navigation | React Navigation 7 | Stack + drawer + tabs |
| Database (Prod) | PostgreSQL | Via DATABASE_URL on Fly.io |
| Database (Dev) | SQLite | backend/database/aqua_track.db fallback |
| Scheduling | APScheduler 3.11 | Asia/Kolkata timezone |
| Reports | pandas, openpyxl, reportlab | Excel and PDF generation |
| Google Integration | gspread, oauth2client | Sheets sync |
| QR / Images | qrcode, Pillow | Bottle QR stickers |
| Container | Docker (python:3.12-slim) | Port 8080 |
| Hosting | Fly.io | aquatrack-prod-final, region: bom |
| Web Hosting | GitHub Pages | gh-pages deploy script |
| Flutter (placeholder) | frontend/pubspec.yaml | Not implemented |

### 3.1 External Services

- Fly.io – Production API hosting (aquatrack-prod-final.fly.dev)
- PostgreSQL – Production database (Fly Postgres attachment assumed)
- Gmail SMTP – OTP email delivery (EMAIL_USER, EMAIL_PASS)
- Google Sheets API – Automated report upload
- Google Play Store – Mobile app distribution
- veekayaquatech.com – Production domain (CORS allowed)
- GitHub Pages – Web portal static hosting

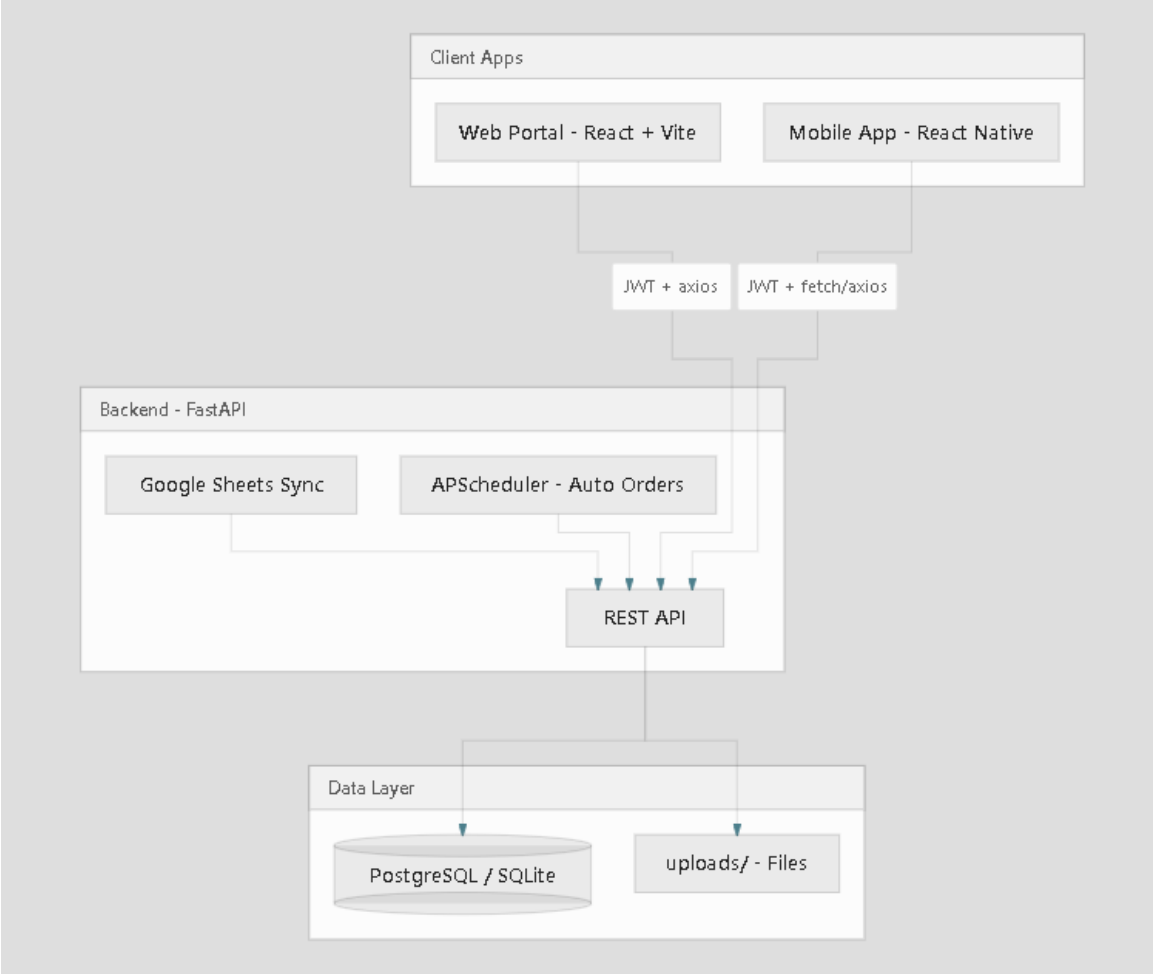
### 3.2 Not Present

- Redis / cache layer
- Message queue (Celery, RabbitMQ, etc.)
- Docker Compose
- Kubernetes manifests
- CI/CD pipelines (.github/workflows)
- Structured monitoring (Datadog, Sentry, etc.)

# 4. Architecture Analysis

## 4.1 Current Architecture

*[Diagram: See Appendix A – System Architecture Diagram]*




Three-tier architecture: Client Layer (Web Portal, Mobile App, Public Pages) → Backend API (FastAPI monolith with in-process scheduler) → Database (PostgreSQL/SQLite) plus local file storage (uploads/). External integrations: Gmail SMTP, Google Sheets API.

## 4.2 Component Responsibilities

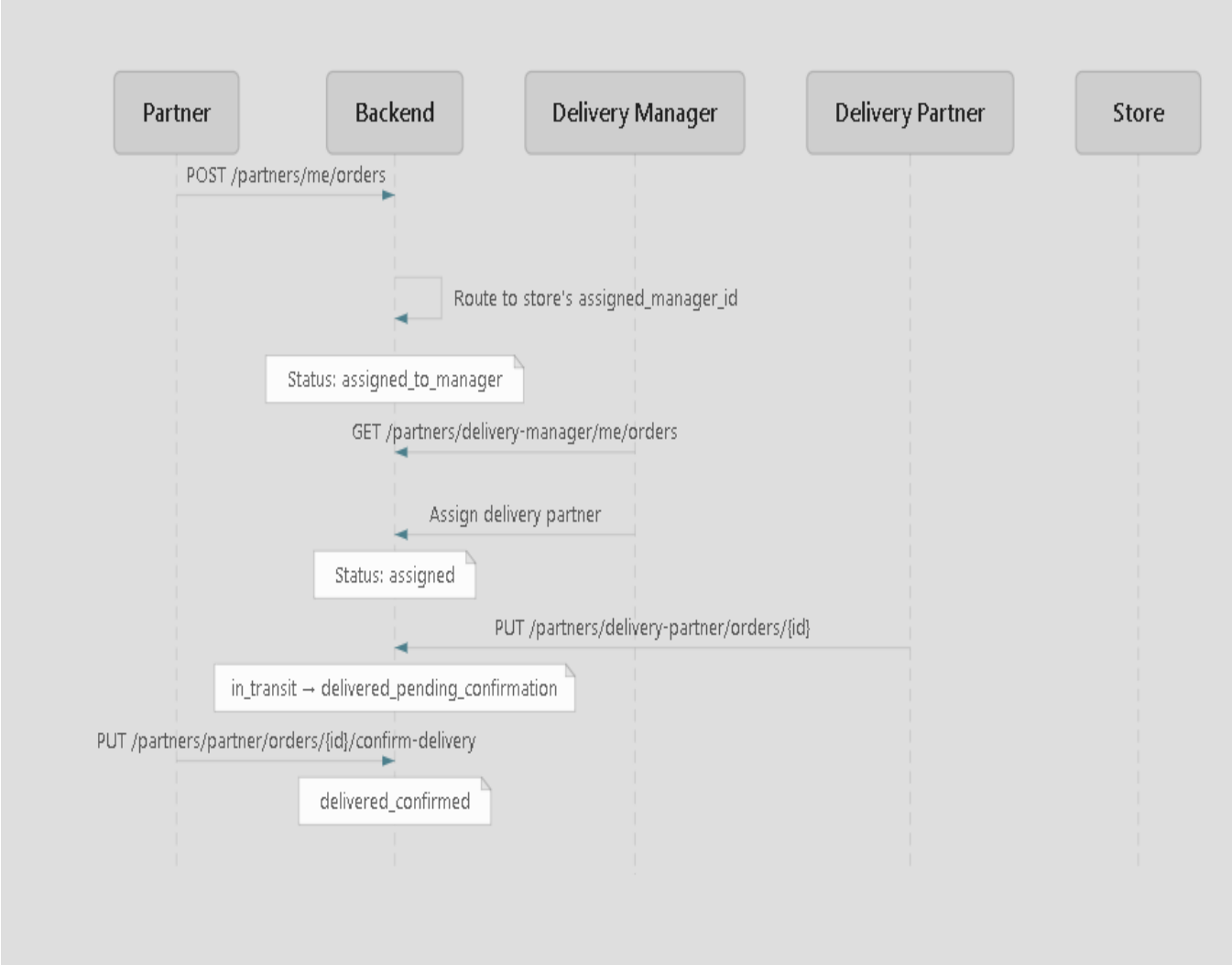
| Component | Responsibility |
|---|---|
| backend/main.py | App entry, CORS, route registration, startup events, static uploads |
| backend/routes/ | HTTP API endpoints grouped by domain |
| backend/models/ | SQLAlchemy ORM entities |
| backend/schemas/ | Pydantic request/response models |
| backend/utils/ | Auth, JWT, auto-order, Google Sheets, config |
| backend/scheduler.py | APScheduler cron configuration |
| web-portal/src/ | React dashboards and auth screens |
| mobile-app/ | React Native field operations app |
| uploads/ | Runtime file storage for photos, compliance docs, reports |

## 4.3 Service Interactions

- All clients authenticate via POST /auth/login and attach JWT Bearer tokens.
- Web portal uses axios/fetch against API_BASE_URL (localhost:8000 dev, Fly.io prod).
- Mobile app uses fetch/axios; config hardcoded to production Fly.io URL.
- On startup, backend runs auto-order immediately and Google Sheet sync in a thread.
- Scheduler runs auto-order daily at 6:00 AM IST.

## 4.4 Data Flow

*[Diagram: See Appendix B – Order Data Flow]*



### 4.5 Integration Points

| Integration | Trigger | Direction |
|---|---|---|
| Gmail SMTP | OTP request / signup | Backend → Gmail |
| Google Sheets | Startup + scheduled logic | Backend → Google API |
| Static uploads | File upload endpoints | Client → Backend → uploads/ |
| Google Play | App version check | Mobile → Play Store URL |

### 4.6 Architectural Strengths

- Clear separation of backend, web, and mobile clients.
- FastAPI provides auto-generated OpenAPI docs (/docs).
- Role-based access control with JWT authentication.
- Rich domain model covering full delivery lifecycle.
- Docker + Fly.io provide reproducible production deployment.
- SQLite fallback enables zero-config local development.

### 4.7 Architectural Weaknesses

- Monolithic backend with no service layer abstraction.
- Background jobs run in-process (scheduler, sheet sync).
- No caching or queue for heavy report generation.
- Doubled API route prefixes create confusion and coupling.
- Single Fly.io machine (no high availability).
- No CI/CD or automated regression testing.
- Large frontend components tightly coupled to API URLs.

## 5. Folder Structure Analysis

### 5.1 Repository Tree

```
aquatrack/
├── backend/                # FastAPI server (core)
│   ├── main.py             # Application entry point
│   ├── scheduler.py        # APScheduler configuration
│   ├── database/           # DB engine, session, SQLite file
│   ├── models/             # SQLAlchemy ORM models (16 files)
│   ├── schemas/            # Pydantic schemas
│   ├── routes/             # API route modules (~25 files)
│   ├── utils/              # Auth, JWT, auto-order, Google Sheets
│   ├── alembic/            # Database migrations (primary)
│   ├── migrations/         # Alternate Alembic setup
│   └── scripts/            # Operational scripts
├── web-portal/             # React admin UI (Vite)
│   └── src/                # Dashboards, auth, config
├── mobile-app/             # React Native field app
│   ├── screens/            # UI screens and utilities
│   ├── android/            # Android native project
│   └── ios/                # iOS native project
├── frontend/               # Placeholder (Flutter – not implemented)
├── docs/                   # Generated documentation (this file)
├── scripts/                # Utility scripts
├── requirements.txt        # Python dependencies
├── Dockerfile              # Container build
├── fly.toml                # Fly.io deployment config
├── alembic.ini             # Root Alembic config
└── migrate.py              # Legacy SQLite→PostgreSQL migration
```

## 5.2 Folder Details

| Folder | Purpose | Responsibility | Dependencies | Improvement Opportunities |
|---|---|---|---|---|
| backend/ | Core API server | FastAPI, SQLAlchemy, scheduler | PostgreSQL/SQLite, external APIs | Extract service layer; consolidate migrations |
| backend/routes/ | HTTP endpoints | models, schemas, utils | Database, file system | Split partners.py; mount delivery_manager router |
| backend/models/ | Data entities | database.db Base | Database schema | Document relationships; add indexes |
| backend/utils/ | Shared utilities | External libs | Env vars, Google creds | Centralize config; structured logging |
| web-portal/ | Admin web UI | Backend API | Node.js, npm | Split SuperAdminDashboard; API client module |
| mobile-app/ | Field mobile app | Backend API | Android SDK, Node 22+ | Build flavors for dev/prod API URL |
| frontend/ | Unused placeholder | None | None | Remove or implement |
| uploads/ | Runtime file storage | Backend routes | Disk/volume | Move to object storage (future) |

## 6. Codebase Assessment

### 6.1 Code Quality Findings

| Finding | Location | Impact |
|---|---|---|
| 8000+ line React component | web-portal/src/SuperAdminDashboard.jsx | Hard to review, test, and modify |
| 1600+ line route file | backend/routes/partners.py | Mixed concerns; high regression risk |
| Triple auth implementations | forgot_password.py, login.py, auth.py | Confusion; only one mounted |
| Hardcoded JWT secret | backend/utils/jwt.py | Security weakness |
| Pydantic v1 orm_mode warnings | backend/schemas/order.py, etc. | Future compatibility |
| print() for logging | 30+ backend files | No log levels or aggregation |
| Broken import path | backend/utils/dependencies.py → ..config | Dead code today; latent bug |

### 6.2 Technical Debt Summary

- Doubled API prefixes (/partners/partners/, /store/store/) – clients depend on exact paths.
- create_superadmin.py uses outdated schema – dangerous if executed.
- Two Alembic migration directories – unclear which is authoritative.
- delivery_manager router imported but not mounted in main.py.
- Mobile role string mismatch: backend uses 'delivery', mobile checks 'delivery_partner'.
- requirements.txt missing pytz, pydantic-settings; emoji breaks Windows pip.

### 6.3 Maintainability Score

**Overall Maintainability Score: 4 / 10**

# 7. Dependency Analysis

## 7.1 Required Software (Windows)

| Software | Version | Purpose |
|---|---|---|
| Git | Latest | Version control |
| Python | 3.11 or 3.12 | Backend runtime |
| Node.js | >= 22.11.0 | Web portal and mobile |
| npm | Bundled with Node | JS package management |
| PostgreSQL | 15+ (optional) | Production-like local DB |
| Android Studio | Latest (optional) | Mobile builds |
| JDK | 17 (optional) | Android builds |
| Docker Desktop | Optional | Container parity with prod |
| Fly CLI | Optional | Production deployment |

## 7.2 Python Dependencies (requirements.txt)

fastapi, uvicorn, python-dotenv, sqlalchemy, pydantic, python-jose, passlib, bcrypt, pandas, openpyxl, gspread, oauth2client, qrcode, pillow, reportlab, requests, apscheduler, psycopg2-binary

**Missing from requirements.txt: pytz, pydantic-settings**

## 7.3 Risks

- Unpinned versions in requirements.txt may cause drift between environments.
- Windows pip install fails on emoji comments without PYTHONUTF8=1.
- Mobile requires Node 22.11+ – older Node versions will fail.

## 8. Database Analysis

### 8.1 Key Entities

| Entity | Table | Primary Key | Description |
|---|---|---|---|
| User | users | id (Integer) | All system users with roles and KYC fields |
| Store | stores | id (String) | Retail/quick-commerce locations |
| Order | orders | id (Integer) | Water delivery orders with status machine |
| Bottle | bottles | uuid (TEXT) | Physical bottle with QR tracking |
| PartnerStore | partner_stores | Composite | Many-to-many partner ↔ store |
| Complaint | complaints | id | Partner complaints with admin assignment |
| StoreComplianceDocument | store_compliance_documents | id | Compliance uploads |
| Attendance | attendance | id | Employee check-in/out records |
| OTP | otps | id | Email OTP for signup/password reset |
| Report | reports | id | Uploaded report files metadata |
| VendorLicense | vendor_licenses | id | State vendor licensing |
| Region | regions | id | Geographic regions |
| Transaction | transactions | id | Bottle movement audit trail |
| Leave | leaves | id | Employee leave requests |
| Holiday | holidays | id | Holiday calendar |

## 8.2 Key Relationships

- User (partner) → Orders placed; User (delivery) → Orders delivered.
- Store → assigned_manager_id → User (delivery_manager).
- Order → assigned_to_manager_id, delivery_person_id, partner_id, store_id.
- Store ↔ User (partner) via partner_stores join table.
- Bottle → assigned to delivery partner or store; linked to orders.
- Complaint → created_by (partner), assigned to admin/channel admin.

## 8.3 Order Status Enum

pending | accepted | assigned_to_manager | assigned | in_transit | delivered_pending_confirmation | delivered_confirmed | cancelled | rejected | delivery_disputed

## 8.4 User Roles

superadmin, admin, vendor, delivery, customer, partner, sub_partner, channel_admin, delivery_manager, employee

## 8.5 Indexes & Performance

- Some foreign keys have index=True (assigned_manager_id, assigned_to_manager_id).
- List endpoints may load full tables without pagination.
- Recommendation: Add indexes on orders.status, orders.order_date, users.email (Phase 3).

## 8.6 Schema Management

- Production: Base.metadata.create_all() on startup + Alembic migrations.
- Two Alembic configs: backend/alembic/ and backend/migrations/ (technical debt).
- Local dev: SQLite auto-created at backend/database/aqua_track.db.

## 9. API Analysis

### 9.1 Route Prefix Map

| Mount Prefix (main.py) | Router Internal Prefix | Example Path |
|---|---|---|
| /auth | none | /auth/login |
| /users | none | /users/signup |
| /partners | /partners | /partners/partners/list |
| /store | /store | /store/store/list/all |
| /orders | none | /superadmin/orders/all |
| /bottle | none | /bottle/superadmin/generate-qr |
| /complaints | /complaints | /complaints/complaints/submit |
| /reports | /reports | /reports/reports/list |
| /attendance | /attendance | /attendance/check-in |
| /store-compliance | /store-compliance | /store-compliance/upload |
| /channel-admin | /channel-admin | /channel-admin/channel-admin/me/dashboard |
| /vendor-license | none | /vendor-license/ |
| /delivery-reports | /delivery-reports | /delivery-reports/orders |
| /app | /app | /app/version |

### 9.2 Authentication

- Unified login: POST /auth/login with { username, password }.
- Returns JWT access_token, user_role, channel_name (channel admins).
- Mobile: requires status='active' for delivery and delivery_manager roles.
- Protected routes use Authorization: Bearer <token> header.
- JWT decoded via get_current_user in utils/jwt.py and utils/auth.py.

### 9.3 Key Endpoint Groups

- Auth: /auth/login, /auth/request-otp, /auth/signup, /auth/verify-*-otp
- Partners: CRUD, order creation, delivery manager/partner endpoints, exports
- Store: CRUD, analytics, auto-order toggle
- Bottle: QR generation, assignment, collection, warehouse
- Reports: upload, list, monthly-pivot/export, download
- Super Admin Orders: /superadmin/orders/all, pending, needs-assignment

Full interactive API documentation available at: http://localhost:8000/docs (when backend is running).

### 9.4 Error Handling

- Standard FastAPI HTTPException with status codes (401, 403, 404, 400).
- Broad except Exception in several routes (report, bottle, auto_order).
- No global exception handler or structured error response format.
- Frontend handles 401 with logout redirect (web and mobile).

## 10. Configuration Analysis

### 10.1 Environment Variables

| Variable | Required | Purpose | Default / Fallback |
|---|---|---|---|
| DATABASE_URL | Prod: Yes | PostgreSQL connection string | SQLite: backend/database/aqua_track.db |
| SECRET_KEY | Recommended | JWT signing key | Hardcoded: YOUR_SECRET_KEY in jwt.py |
| EMAIL_USER | For OTP | Gmail sender address | None – OTP emails fail |
| EMAIL_PASS | For OTP | Gmail app password | None |
| PORT | Optional | Server port | 8080 (main.py); use 8000 for web dev |
| VITE_API_BASE_URL | Web prod build | API URL for web portal | aquatrack-prod-final.fly.dev |

### 10.2 Configuration Files

| File | Purpose |
|---|---|
| .env (gitignored) | Local environment variables |
| fly.toml | Fly.io app config (region bom, 2GB RAM, port 8080) |
| Dockerfile | Python 3.12 container build |
| web-portal/src/config.js | API base URL by environment |
| mobile-app/src/config.ts | Mobile API URL (hardcoded prod) |
| backend/utils/google_service.json | Google service account credentials |
| alembic.ini | Root Alembic config pointing to backend/migrations |

### 10.3 Secrets Management

Current state: Secrets are partially managed via Fly.io secrets (assumed for production). Local development requires manual .env creation. Several secrets are committed to the repository (google_service.json, migrate.py credentials) – these must be rotated.

## 11. Local Development Setup Guide (Windows)

### 11.1 Prerequisites

- Git
- Python 3.11+
- Node.js 22.11+
- Optional: PostgreSQL, Docker Desktop, Android Studio

### 11.2 Backend Setup

1. 1. cd E:\Development\git\repository\veekayaquatech\Services\aquatrack
2. 2. python -m venv .venv
3. 3. .venv\Scripts\Activate.ps1
4. 4. $env:PYTHONUTF8 = "1"
5. 5. pip install -r requirements.txt
6. 6. pip install pytz pydantic-settings
7. 7. Create .env file (see Section 10.1)
8. 8. uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000

### 11.3 Seed Super Admin

Do NOT use backend/create_superadmin.py (outdated schema). Use Python script from Appendix D.

### 11.4 Web Portal Setup

9. 1. cd web-portal
10. 2. npm install
11. 3. npm run dev
12. 4. Open http://localhost:5173/login

### 11.5 Mobile Setup (Optional)

- Edit mobile-app/src/config.ts to point to local API (10.0.2.2:8000 for emulator).
- cd mobile-app && npm install
- npm start (Metro) + npm run android

### 11.6 Verification

| Check | URL / Command | Expected Result |
|---|---|---|
| API root | http://127.0.0.1:8000/ | Welcome message JSON |
| OpenAPI docs | http://127.0.0.1:8000/docs | Swagger UI |
| Login test | POST /auth/login | access_token returned |
| Web login | http://localhost:5173/login | Dashboard redirect |

### 11.7 Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| ModuleNotFoundError: backend | Wrong working directory | Run uvicorn from project root |
| pip UnicodeDecodeError | Emoji in requirements.txt | Set PYTHONUTF8=1 |
| 401 on all requests | Invalid/expired JWT | Re-login |
| Web CORS error | Backend not running or wrong port | Start backend on port 8000 |
| Slow startup | Auto-order + Google Sheet sync | Expected; disable for local dev (future flag) |
| DM dashboard 404 | delivery_manager router not mounted | Use /partners/partners/delivery-manager/... or mount router |

## 12. Deployment Analysis

### 12.1 Current Deployment Process

- Backend: fly deploy from repo root (uses Dockerfile + fly.toml).
- Fly secrets: DATABASE_URL, EMAIL_USER, EMAIL_PASS (assumed).
- Web: cd web-portal && npm run build && npm run deploy (gh-pages).
- Mobile: Manual release build via Android Studio → Google Play.

### 12.2 CI/CD Pipelines

None configured. All deployments are manual.

### 12.3 Rollback Process

- Fly.io: fly releases list → fly deploy --image <previous-image>
- Web: Revert git commit and redeploy gh-pages
- Database: No automated rollback; rely on Fly Postgres backups

### 12.4 Deployment Risks

- No automated tests before deploy
- Startup side effects (auto-order, sheet sync) on every deploy
- Single machine – deploy causes brief downtime
- No health check endpoint for Fly (recommend adding /health)

## 13. Monitoring & Logging Review

### 13.1 Existing Logging

- print() statements throughout backend (~30+ files)
- Fly.io platform logs via fly logs
- Console.log in web portal and mobile app
- Alembic logging.ini for migrations

### 13.2 Missing Logging & Monitoring

- Structured logging (JSON) with log levels
- Request/response logging middleware
- Error tracking (Sentry or equivalent)
- Uptime monitoring and alerting
- Metrics (request latency, error rates, scheduler success)
- Database query performance monitoring

### 13.3 Recommendations

| Recommendation | Risk | Effort | Priority |
|---|---|---|---|
| Replace print() with Python logging module | None | 4–6 hours | P1 |
| Add GET /health endpoint | None | 1–2 hours | P1 |
| Configure Fly.io health checks on /health | Very Low | 1 hour | P2 |
| Add Sentry for error tracking | Low | 1 day | P3 |
| Uptime monitoring (UptimeRobot, etc.) | None | 2 hours | P2 |

## 14. Security Review

| Issue | Severity | Location | Recommendation |
|---|---|---|---|
| JWT secret hardcoded | Critical | backend/utils/jwt.py | Load from env; rotate in maintenance window |
| Google service account in repo | Critical | backend/utils/google_service.json | Rotate; move to Fly secret/volume |
| DB credentials in migrate.py | Critical | migrate.py | Remove; rotate password |
| No rate limiting on login/OTP | Medium | /auth/* | Phase 4 – add rate limiting |
| Static /uploads without auth | Medium | main.py | Audit; add auth proxy if needed |
| OTP debug logging | Low | otp_utils.py | Remove debug prints |
| CORS credentials enabled | Low | main.py | Document; do not broaden origins |

### 14.1 Authentication & Authorization

- Authentication: Email + bcrypt password → JWT token.
- Authorization: Role checks per endpoint (require_role, inline role checks).
- Field roles (delivery, delivery_manager) require status='active'.
- No fine-grained permission system beyond role enum.

## 15. Performance Review

| Area | Issue | Impact | Safe Improvement |
|---|---|---|---|
| Startup | Auto-order scans all stores on boot | Slow cold start | Env flag to skip on local dev |
| Startup | Google Sheet sync thread | Errors/noise on boot | Schedule-only; env flag |
| Reports | pandas Excel in request path | High latency on large datasets | Background job (Phase 3+) |
| List APIs | No pagination on some endpoints | Memory/latency growth | Add pagination (Phase 3 – breaking if clients expect all) |
| Database | Missing indexes on hot columns | Slow queries at scale | Add indexes via migration |
| Architecture | Single process | No horizontal scaling of workers | Separate worker process (Phase 4) |

**Scalability Score: 3 / 10**

## 16. Refactoring Recommendations

**Critical constraint: The application is production-active. All recommendations preserve existing API paths, response shapes, and business logic unless explicitly noted.**

### Phase 1 – Zero Risk

| Description | Benefits | Risk | Effort | Dependencies | Priority | No Behavior Change? |
|---|---|---|---|---|---|---|
| Add .env.example + RUNBOOK.md | Faster onboarding | None | 4h | None | P0 | YES |
| Fix requirements.txt (deps + no emoji) | Reproducible installs | None | 1h | None | P0 | YES |
| Replace print() with logging | Better ops visibility | None | 6h | None | P1 | YES |
| Add /health endpoint | Deploy monitoring | None | 2h | None | P1 | YES |
| Document API URL map | Prevent breaking changes | None | 4h | None | P1 | YES |
| Mark deprecated dead code | Prevent misuse | None | 1h | None | P2 | YES |
| Fix Pydantic orm_mode warnings | Clean startup | None | 30m | None | P2 | YES |
| Remove OTP debug logging | Security hygiene | None | 30m | None | P1 | YES |

## Phase 2 – Very Low Risk

| Description | Benefits | Risk | Effort | Dependencies | Priority | No Behavior Change? |
|---|---|---|---|---|---|---|
| Centralize JWT config (same default secret) | Env-based config | Very Low | 2h | None | P2 | YES |
| Mount delivery_manager router | Fix web DM dashboard | Very Low | 1h | Smoke test | P2 | YES |
| Env-gated background jobs (default on) | Faster local dev | Very Low | 2h | None | P3 | YES |
| Extract OTP email helper | DRY | Very Low | 3h | None | P3 | YES |
| Split SuperAdminDashboard (extract only) | Maintainability | Very Low | 3–5d | None | P2 | YES |
| Frontend API client module | Maintainability | Very Low | 2–3d | None | P2 | YES |
| pytest smoke tests | Regression safety | Very Low | 1d | None | P2 | YES |
| Migrate to FastAPI lifespan | Future compatibility | Very Low | 2h | None | P3 | YES |

## Phase 3 – Medium Risk

| Description | Benefits | Risk | Effort | Dependencies | Priority | No Behavior Change? |
|---|---|---|---|---|---|---|
| Database indexes on hot columns | Query performance | Medium | 1d | Staging test | P3 | YES |
| Optimize N+1 queries (joinedload) | Latency reduction | Medium | 2–3d | Staging test | P3 | YES |
| Move sheet sync to scheduler only | Predictable startup | Medium | 1d | Business sign-off | P4 | NO – timing change |
| Rotate JWT secret (maintenance) | Security | Medium | 4h | User comms | P2 | NO – sessions invalidated |

### Phase 4 – Long-Term

| Description | Benefits | Risk | Effort | Dependencies | Priority | No Behavior Change? |
|---|---|---|---|---|---|---|
| Separate background worker process | Scalability | High | 2–4w | Infrastructure | P4 | NO |
| Redis cache layer | Performance | Medium–High | 2–3w | Cache invalidation design | P4 | NO |
| Object storage for uploads | Scalability | Medium | 1–2w | S3/GCS setup | P4 | NO |
| CI/CD pipeline | Quality gates | Low–Medium | 1w | GitHub Actions | P3 | YES |
| API versioning (/v1/) | Future evolution | High | 4w+ | Client updates | P5 | NO |
| Normalize doubled URL prefixes | Clean URLs | High | N/A | All clients | Never | NO – BREAKING |

## 17. Proposed Future Architecture

### 17.1 Recommended Backend Structure

```
backend/
├── main.py
├── config/              # Centralized settings (env-based)
├── api/
│   └── routes/          # Thin route handlers
├── services/            # Business logic layer
├── repositories/        # Data access layer (optional)
├── models/
├── schemas/
├── workers/             # Background jobs (future)
└── tests/
```

### 17.2 Separation of Concerns

- Routes: HTTP parsing, auth checks, response formatting only.
- Services: Order routing, auto-order logic, report generation.
- Workers: Scheduler, Google Sheet sync, heavy exports.
- Config: Single Settings class loaded from environment.

### 17.3 Scalability Path

- Phase A: Add /health, logging, smoke tests (no architecture change).
- Phase B: Extract service layer behind existing routes.
- Phase C: Move scheduler + reports to separate Fly process.
- Phase D: Object storage + CDN for uploads.
- Phase E: Read replica or connection pooling tuning.

## 18. Knowledge Transfer Section

### 18.1 Important Business Rules

- Orders auto-route to store's assigned_manager_id (delivery manager).
- Auto-order requires: store.auto_order_enabled, assigned manager, mapped partner.
- Auto-order runs daily at 6:00 AM IST and on every server startup.
- Delivery partners must have status='active' before mobile login.
- Channels: BLINKIT, ZEPTO, IBM, GENERAL, CUSTOM – affect reporting and routing.
- Monthly cap logic applies to ZEPTO channel consumption.
- Bottle price reference: ₹100 per bottle (SuperAdminDashboard constant).

### 18.2 Hidden Assumptions

- API paths with doubled prefixes are intentional – frontends hardcode them.
- Mounted auth router is forgot_password.py at /auth – not login.py or auth.py.
- JWT secret 'YOUR_SECRET_KEY' may be the actual production secret if never changed.
- Mobile app points to production API by default in config.ts.
- Web dev expects backend on port 8000; Docker/Fly use 8080.
- create_all() runs on every startup – migrations may not be sole schema source.

### 18.3 Operational Procedures

- Approve delivery partner: Super Admin → Delivery Partners → Approve.
- Map store to manager: Super Admin → Stores → Assign manager.
- Map partner to store: Super Admin → Partners → Assign stores.
- Production logs: fly logs -a aquatrack-prod-final.
- Production restart: fly apps restart aquatrack-prod-final.

### 18.4 Frequently Modified Areas

- backend/routes/partners.py – largest route module
- web-portal/src/SuperAdminDashboard.jsx – largest UI file
- backend/utils/auto_order.py and auto_order_runner.py – auto-order logic
- backend/routes/forgot_password.py – login and OTP
- backend/main.py – route registration and startup

## 19. Team Review Checklist

- ☐Confirm production URL and Fly.io app name (aquatrack-prod-final)
- ☐Verify who has Fly.io, GitHub, and Google Cloud access
- ☐Confirm production DATABASE_URL and backup strategy
- ☐Verify JWT SECRET_KEY value in production (is it still default?)
- ☐Confirm Google service account is valid and sheet ID is correct
- ☐Verify Gmail OTP credentials are active
- ☐Review who uses each role (superadmin, partner, DM, DP, channel admin)
- ☐Confirm mobile app production API URL in config.ts
- ☐Verify web portal VITE_API_BASE_URL for production builds
- ☐Review Fly.io machine size and scaling needs
- ☐Confirm uploads/ directory persistence strategy on Fly
- ☐Identify critical business periods (avoid deploys during peak delivery)
- ☐Assign owner for secret rotation plan
- ☐Agree on Phase 1 improvements to implement first
- ☐Schedule regression test plan before any Phase 3 changes
- ☐Document production super admin contact credentials (secure vault)
- ☐Review open complaints and order backlog as health indicators

## 20. Action Plan

### 20.1 Immediate Actions (Week 1)

| Action | Owner | Effort |
|---|---|---|
| Create .env.example and RUNBOOK.md | Dev Lead | 4h |
| Fix requirements.txt | Developer | 1h |
| Add /health endpoint | Developer | 2h |
| Verify backend starts locally on Windows | Developer | 2h |
| Seed local super admin for testing | Developer | 30m |
| Inventory production secrets (Fly dashboard) | DevOps | 2h |

### 20.2 Next 30 Days

- Implement structured logging (replace print statements).
- Add pytest smoke test suite and run before deploys.
- Centralize JWT config (preserve default secret value).
- Mount delivery_manager router and verify DM dashboard.
- Document complete API URL map from OpenAPI spec.
- Mark deprecated files; remove credentials from migrate.py.
- Begin SuperAdminDashboard extract-refactor (no logic changes).

### 20.3 Next 90 Days

- Set up GitHub Actions CI (install + smoke tests).
- Create staging environment on Fly.io.
- Add database indexes on staging; validate; apply to prod.
- Rotate JWT secret and Google service account (planned maintenance).
- Frontend API client module across all dashboards.
- Env-gated background jobs for dev/prod parity.

### 20.4 Long-Term Roadmap (6+ Months)

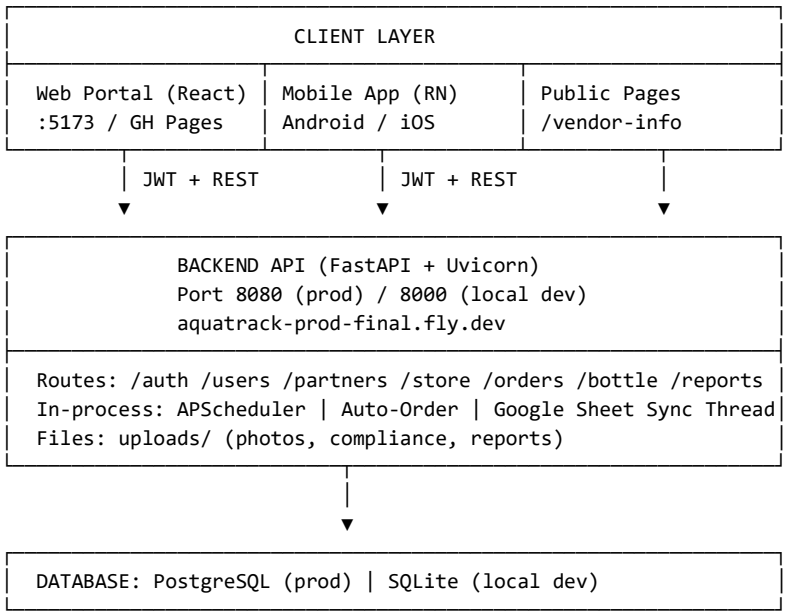
- Separate background worker process for scheduler and reports.
- Comprehensive integration test suite (order lifecycle).
- Object storage migration for uploads.
- Observability stack (structured logs + error tracking + uptime).
- Incremental service layer extraction in backend.
- Mobile build flavors (dev/staging/prod).

## 21. Risk Register

| ID | Risk | Impact | Probability | Mitigation |
|---|---|---|---|---|
| R01 | JWT secret compromise | Critical | Medium | Rotate secret; load from env; audit access |
| R02 | Committed Google credentials | Critical | High | Rotate service account; use Fly secrets |
| R03 | Single Fly.io machine failure | High | Medium | Add health checks; consider second machine |
| R04 | No CI/CD – bad deploy to prod | High | Medium | Smoke tests; staging env; manual checklist |
| R05 | Large file refactor regression | High | Medium | Extract-only refactors; smoke tests first |
| R06 | Schema drift (create_all vs Alembic) | Medium | Medium | Consolidate migration strategy |
| R07 | Startup auto-order on empty/wrong DB | Medium | Low | Env flag; validate DB before order run |
| R08 | Breaking API path 'cleanup' | Critical | Low | Document doubled prefixes; code review gate |
| R09 | No database backup verification | Critical | Medium | Verify Fly Postgres backup schedule |
| R10 | Key person dependency | High | High | This document + knowledge transfer sessions |

# 22. Appendix

## Appendix A – System Architecture Diagram




EXTERNAL: Gmail SMTP | Google Sheets API | Google Play Store

## Appendix B – Order Data Flow

```
Partner → POST /partners/partners/me/orders
       → Order created (status: assigned_to_manager)
       → assigned_to_manager_id = store.assigned_manager_id

Delivery Manager → GET /delivery-manager/me/orders (or /partners/partners/delivery-
manager/me/orders)
                → Assigns Delivery Partner

Delivery Partner → PUT /partners/partners/delivery-partner/orders/{id}
                → Status: in_transit → delivered_pending_confirmation

Partner → PUT /partners/partners/partner/orders/{id}/confirm-delivery
       → Status: delivered_confirmed
```

## Appendix C – Useful Commands

| Purpose | Command |
|---|---|
| Backend (dev) | uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000 |
| Web portal (dev) | cd web-portal && npm run dev |
| Mobile Metro | cd mobile-app && npm start |
| Mobile Android | cd mobile-app && npm run android |
| Fly logs | fly logs -a aquatrack-prod-final |
| Fly deploy | fly deploy |
| Fly secrets | fly secrets list -a aquatrack-prod-final |
| Docker build | docker build -t aquatrack-api . |
| Test backend import | python -c "from backend.main import app; print(app.title)" |

## Appendix D – Seed Super Admin Script

```
python -c "
from backend.database.db import SessionLocal
from backend.models.user import User, RoleEnum
from backend.utils.auth import hash_password
db = SessionLocal()
email = 'admin@local.test'
if not db.query(User).filter(User.email == email).first():
    db.add(User(email=email, password=hash_password('Admin@12345'),
        full_name='Local Super Admin', role=RoleEnum.superadmin, status='active'))
    db.commit()
    print('Created:', email)
else:
    print('Already exists:', email)
db.close()
"
```

## Appendix E – Illustration Placeholders

- [Screenshot Placeholder: Super Admin Dashboard – Orders Tab]
- [Screenshot Placeholder: FastAPI /docs Swagger UI]
- [Screenshot Placeholder: Mobile App – Delivery Partner Dashboard]
- [Screenshot Placeholder: Fly.io Dashboard – aquatrack-prod-final]
- [Screenshot Placeholder: Order Status Flow Diagram (Visio/Lucidchart)]

## Document Control

| Version | Date | Author | Changes |
|---|---|---|---|
| 1.0 | 2026-06-05 | Architecture Review Team | Initial comprehensive documentation |